Meta Quest Red-Block Teleoperation Operator Guide

Installation · launch profiles · runtime settings · architecture · operations

Verified simulator workflow

Seven red-block tasks · Meta Quest via Unitree xr_teleoperate · DDS control · ZMQ video

Document date
2026-08-21

Repository branch
fix/teleop-randomized-hospital-integration

Important: isolate the simulator DDS domain from physical robots unless commanding them is intentional.

Purpose and verified scope

This guide explains how to run every supported red-block task in core_unitree_sim_isaaclab with Meta Quest glasses through Unitree xr_teleoperate. It covers installation, networking, launch profiles, simulator and XR settings, DDS topics, camera transport, resets, recording, troubleshooting, and the architecture behind the complete control loop.

The supported path is Meta Quest → xr_teleoperate → DDS → Isaac Lab for control and Isaac RTX cameras → shared memory → teleimager/ZMQ → xr_teleoperate/Vuer → Meta Quest for vision. Use the repository's --meta_quest switch; do not use Isaac Sim's unrelated native --xr mode.

> Safety boundary: DDS domain 1 carries robot commands. Keep real robots off the simulator network/domain unless they are intentionally meant to receive the same commands. Align the operator's arms with the simulated robot's initial pose before enabling control.

What was verified

- All seven registered red-block tasks started headlessly on 2026-08-21 with Isaac Sim 5.1, Isaac Lab, CUDA/Vulkan, and an NVIDIA RTX PRO 6000 Blackwell.
- Each task constructed its scene, robot, end effector, and three 480x640 RTX cameras; opened ZMQ publishers; registered the expected DDS endpoints; accepted the DDS action source; ran bounded control steps; and exited with status 0.
- The randomized hospital geometry suite passed 1,000 deterministic layouts.
- Static Quest-profile tests verify task mappings and reject incompatible launch settings.
- A physical Quest and an external xr_teleoperate host were not available in this workspace. The browser handshake, headset tracking packets, and human-in-the-loop manipulation remain hardware validation items.

System roles

- Simulator host: this repository, Isaac Sim/Isaac Lab, scene physics, robot controller, DDS bridge, cameras, and teleimager ZMQ server.
- XR host: Unitree xr_teleoperate, Vuer HTTPS/WSS server, Meta Quest pose receiver, inverse kinematics, hand retargeting, DDS publishers, and camera client.
- Meta Quest: Browser/Vuer display plus hand tracking or controllers.

Prerequisites

Simulator host

- A working checkout of Cognitive-Software-Labs/core_unitree_sim_isaaclab on the integration branch.
- The repository's Isaac Sim/Isaac Lab environment and NVIDIA driver stack.
- Robot, room, and object assets. From the repository root, use . fetch_assets.sh if assets/ is absent or incomplete.
- A reachable IPv4 address and a network interface that can carry both ZMQ traffic and CycloneDDS multicast.
- GPU execution is recommended for three real-time RTX cameras. CPU is acceptable for diagnostics but may not sustain the desired control and image rates.

XR host and headset

- Ubuntu 20.04 or 22.04 is the upstream tested host baseline for xr_teleoperate.
- Meta Quest 3 or another Quest browser capable of WebXR.
- The Quest and XR host on the same reachable network. The XR host must also reach the simulator host.
- A TLS certificate for the Vuer browser endpoint on the XR host.
- Unitree unitree_sdk2_python for DDS communication.

Discover network values

Run this on both Linux hosts and record the simulator IP, XR-host IP, and interface name such as enp5s0, eth0, or wlan0:

```
ip -br addr
```

Use the simulator IP for --img-server-ip. Use the XR-host IP in the Quest browser URL. Use the correct shared-network interface for --network-interface so CycloneDDS does not bind to a VPN, Docker bridge, or loopback device.

A single machine may run the simulator and xr_teleoperate. In that case use its LAN IP, not 127.0.0.1, in the Quest URL because the headset must reach it.

Install Unitree xr_teleoperate on the XR host

The following is the upstream installation flow for xr_teleoperate v1.6, current on 2026-08-21. Keep this environment separate from the Isaac Sim environment.

```
conda create -n tv python=3.10 pinocchio=3.1.0 numpy=1.26.4 -c conda-forge
conda activate tv

git clone https://github.com/unitreerobotics/xr_teleoperate.git
cd xr_teleoperate
git submodule update --init --depth 1

cd teleop/teleimager
pip install -e . --no-deps

cd ../televuer
pip install -e .
```

Install the current Unitree Python SDK separately:

```
cd ~
git clone https://github.com/unitreerobotics/unitree_sdk2_python.git
cd unitree_sdk2_python
pip install -e .
```

For xr_teleoperate v1.1 or later, upstream requires unitree_sdk2_python commit 404fe44d76f705c002c97e773276f2a8fefb57e4 or newer.

Create the Quest/Vuer certificate

Run from the xr_teleoperate/teleop/televuer directory:

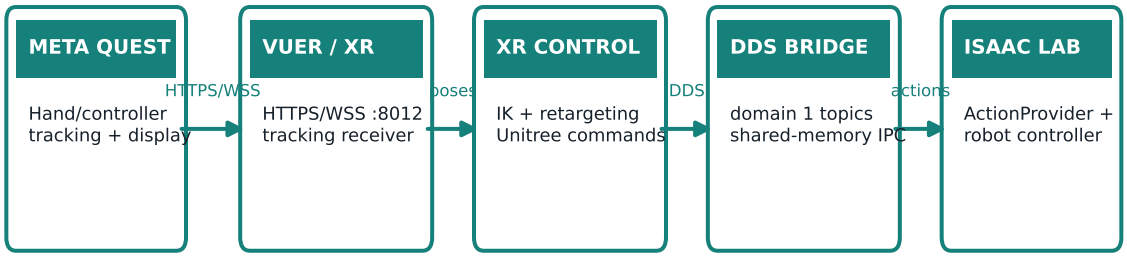
```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout key.pem -out cert.pem

mkdir -p ~/.config/xr_teleoperate
cp cert.pem key.pem ~/.config/xr_teleoperate/

sudo ufw allow 8012
```

The certificate secures the XR host's Vuer HTTPS/WSS endpoint. The simulator-side --meta_quest mode disables teleimager's optional direct WebRTC server, so teleimager does not need a second certificate for this path.

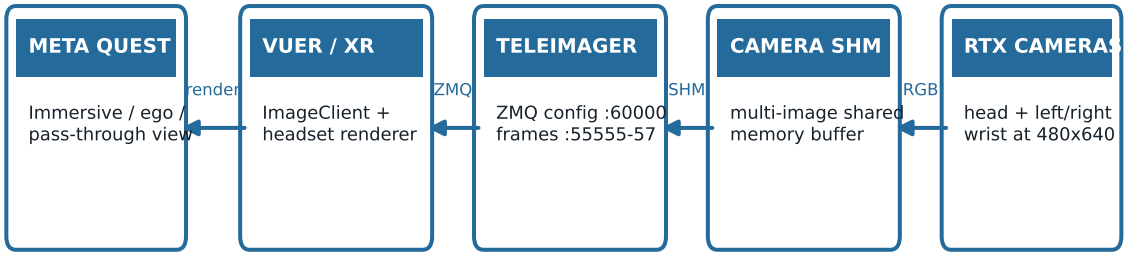
End-to-end architecture

Control travels from the headset to Isaac Sim; rendered camera frames return by a separate path.

CONTROL PATH · Quest → simulator



VIDEO PATH · simulator → Quest



Why there are two servers

The simulator exposes camera frames through teleimager/ZMQ. xr_teleoperate consumes those frames and hosts the browser-facing Vuer session. --meta_quest disables teleimager direct WebRTC, but Vuer still needs its own TLS certificate.

Architecture details

Control path: Quest to simulator

1. The Quest browser connects to Vuer on the XR host over HTTPS/WSS port 8012 and provides hand or controller poses.
2. teleop_hand_and_arm.py converts the XR poses into robot arm targets through inverse kinematics and converts hand poses into Dex1, Dex3, or Inspire commands through retargeting.
3. Unitree SDK2 publishes the arm and hand commands on CycloneDDS domain 1.
4. The simulator DDS processes subscribe to the topics and copy commands into named shared-memory buffers.
5. The selected DDS ActionProvider reads those buffers. The layered controller applies the action to the Isaac Lab robot at the configured control rate.
6. Joint/hand state returns through shared memory and DDS to support closed-loop operation.

Video path: simulator to Quest

1. --meta_quest forces front_camera, left_wrist_camera, and right_wrist_camera on and keeps RTX rendering active.
2. The simulator writes current RGB observations to teleimager's multi-image shared-memory buffer every control step by default.
3. Teleimager exposes configuration on port 60000 and publishes head, left-wrist, and right-wrist frames on ZMQ ports 55555, 55556, and 55557.
4. xr_teleoperate connects its image client to the simulator IP and passes those images into Vuer.
5. Vuer renders the selected immersive, ego, or pass-through presentation in the Quest browser.

Why --meta_quest disables direct WebRTC

There are two browser-capable layers in the wider Unitree stack. In this repository's supported mode, the browser connects to Vuer on the XR host, while the XR host consumes simulator cameras over ZMQ. Starting teleimager's direct WebRTC ports 60001-60003 would be redundant and can block startup on missing certificates. Therefore --meta_quest sets TELEIMAGER_DISABLE_WEBRTC=1.

> Do not browse to the simulator's port 60001 for this workflow. Browse directly to the XR host's Vuer URL on port 8012.

Supported task and XR profile matrix

G1 29-DoF with Dex1

- Standard: Isaac-PickPlace-RedBlock-G129-Dex1-Joint
- Randomized hospital: Isaac-PickPlace-RedBlock-Hospital-G129-Dex1-Joint
- XR profile: --arm=G1_29 --ee=dex1
- Input: hand tracking or Quest controllers.

G1 29-DoF with Dex3

- Pick/place: Isaac-PickPlace-RedBlock-G129-Dex3-Joint
- Drawer: Isaac-Pick-Redblock-Into-Drawer-G129-Dex3-Joint
- XR profile: --arm=G1_29 --ee=dex3
- Input: hand tracking. Upstream xr_teleoperate does not support controller input for Dex3.

G1 29-DoF with Inspire DFX

- Task: Isaac-PickPlace-RedBlock-G129-Inspire-Joint
- XR profile: --arm=G1_29 --ee=inspire_dfx
- Input: hand tracking.
- Important: use inspire_dfx, not inspire_ftp. This simulator's rt/inspire/cmd and rt/inspire/state topics match the DFX controller; the FTP profile uses different topics.

H1-2 27-DoF with Inspire DFX

- Task: Isaac-PickPlace-RedBlock-H12-27dof-Inspire-Joint
- XR profile: --arm=H1_2 --ee=inspire_dfx
- Input: hand tracking.

G1 29-DoF with Dex1 drawer

- Task: Isaac-Pick-Redblock-Into-Drawer-G129-Dex1-Joint
- XR profile: --arm=G1_29 --ee=dex1
- Input: hand tracking or Quest controllers.

--meta_quest deliberately uses an explicit allowlist. An unverified task fails early instead of silently starting with the wrong robot, hand, or camera mapping.

Launch the simulator

From the repository root, start one task in its own terminal. Recommended GPU/headless example:

```
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-PickPlace-RedBlock-G129-Dex1-Joint
```

For a local Isaac Sim window, omit --headless. Headless still renders offscreen camera frames. Never replace --headless with --no_render; Quest cameras require rendering and the launch guard rejects that combination.

All seven simulator commands

```
# G1 + Dex1
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-PickPlace-RedBlock-G129-Dex1-Joint

# G1 + Dex1, randomized hospital
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-PickPlace-RedBlock-Hospital-G129-Dex1-Joint

# G1 + Dex3
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-PickPlace-RedBlock-G129-Dex3-Joint

# G1 + Inspire DFX
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-PickPlace-RedBlock-G129-Inspire-Joint
```

```
# H1-2 + Inspire DFX
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-PickPlace-RedBlock-H12-27dof-Inspire-Joint

# Drawer, G1 + Dex1
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-Pick-Redblock-Into-Drawer-G129-Dex1-Joint

# Drawer, G1 + Dex3
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-Pick-Redblock-Into-Drawer-G129-Dex3-Joint
```

Do not manually add --robot_type or an --enable_*_dds hand flag. Quest mode derives them from the selected task and rejects conflicting hand flags.

Launch xr_teleoperate

Open a second terminal on the XR host. Activate the tv environment and enter the teleop directory:

```
conda activate tv
cd ~/xr_teleoperate/teleop
```

Replace SIM_IP and IFACE in one matching command. The --sim flag is mandatory: without it, xr_teleoperate follows its physical-robot startup path.

```
# G1 + Dex1: controller input
python teleop_hand_and_arm.py --input-mode=controller \
  --arm=G1_29 --ee=dex1 --sim \
  --img-server-ip=SIM_IP --network-interface=IFACE

# G1 + Dex1: hand tracking
python teleop_hand_and_arm.py --input-mode=hand \
  --arm=G1_29 --ee=dex1 --sim \
  --img-server-ip=SIM_IP --network-interface=IFACE

# G1 + Dex3: hand tracking
python teleop_hand_and_arm.py --input-mode=hand \
  --arm=G1_29 --ee=dex3 --sim \
  --img-server-ip=SIM_IP --network-interface=IFACE
```

```
# G1 + Inspire DFX: hand tracking
python teleop_hand_and_arm.py --input-mode=hand \
  --arm=G1_29 --ee=inspire_dfx --sim \
  --img-server-ip=SIM_IP --network-interface=IFACE

# H1-2 + Inspire DFX: hand tracking
python teleop_hand_and_arm.py --input-mode=hand \
  --arm=H1_2 --ee=inspire_dfx --sim \
  --img-server-ip=SIM_IP --network-interface=IFACE
```

Add --record when episode recording is wanted. Add --headless to xr_teleoperate only when its host has no local display; this is independent of the simulator's --headless setting.

Connect the Meta Quest and start control

1. Wait until the simulator reports the three cameras ready, image-server startup, DDS registration, ActionProvider creation, and controller started, start main loop....
2. Wait until xr_teleoperate has connected to the simulator image server and started Vuer.
3. Put the Quest on the same network as the XR host.
4. Open the Quest browser at the URL below, replacing XR_HOST_IP with the XR host's LAN address.

```
https://XR_HOST_IP:8012/?ws=wss://XR_HOST_IP:8012
```

5. For a self-signed certificate warning, select Advanced and proceed to the address. This trust step is normally required once per certificate.
6. In Vuer, select Virtual Reality and allow the requested WebXR/headset permissions.
7. Confirm that the robot's head-camera view appears. Verify wrist views if the selected Vuer mode presents them.
8. Stand in a clear area. Align your arms with the robot's displayed initial pose to avoid a command discontinuity.
9. In the xr_teleoperate terminal, press r to begin teleoperation.
10. Press q in the xr_teleoperate terminal to quit. Stop the simulator with Ctrl+C after the XR process has shut down.

Recording

Start xr_teleoperate with --record. After pressing r, press s to start an episode and press s again to stop and save it. Repeat s for additional episodes. Upstream defaults to xr_teleoperate/teleop/utils/data; monitor disk capacity when recording three camera streams.

Optional recording metadata includes --task-dir, --task-name, --task-goal, --task-desc, and --task-steps. The upstream --frequency default is 30 Hz and controls both teleoperation and recording cadence.

> In simulation mode, xr_teleoperate may publish an object reset after saving an episode. This repository handles that as reset category 1, preserving the current randomized room layout.

Simulator run settings

Settings normally supplied by the operator

- --task TASK_ID: selects one exact registered task from the profile matrix.
- --meta_quest: applies and validates the Quest integration profile.
- --device cuda:0: recommended simulation device. Use cpu for diagnostics or where GPU simulation is unavailable.
- --headless: removes the local window while keeping offscreen RTX rendering active.
- --seed 42: environment/randomizer seed; default 42.
- --max_steps N: clean bounded run for smoke tests. Omit for an interactive session.
- --stats_interval 10.0: seconds between runtime statistics; default 10.

Settings forced or inferred by --meta_quest

- num_envs = 1: teleoperation operates one scene and does not silently control environment index 0 among multiple environments.
- action_source = dds: live arm and hand commands come from DDS-backed action providers.
- robot_type = g129 or h1_2: derived from the task.
- One of enable_dex1_dds, enable_dex3_dds, or enable_inspire_dds: derived from the task.
- enable_cameras = True.
- camera_include = front_camera,left_wrist_camera,right_wrist_camera.
- camera_write_interval = 1 unless explicitly supplied.
- TELEIMAGER_DISABLE_WEBRTC = 1; ZMQ remains enabled.

Performance and physics tuning

- --step_hz 100: controller target rate, default 100 Hz.
- --physics_dt SECONDS: overrides physics timestep; task default applies when omitted.
- --render_interval N: render once every N simulation steps; do not increase without checking headset latency.
- --camera_write_interval N: publish camera shared memory every N control steps. Quest mode defaults to 1.
- --camera_jpeg_quality 85: JPEG quality from 1-100; default 85.
- --camera_include LIST and --camera_exclude LIST: camera filters. Quest mode needs all three required sensors.
- --solver_iterations N, --physx_substeps N, --gravity_z VALUE: low-level physics overrides; leave task defaults unless investigating stability.
- --profile_interval 500: profiling report interval in control steps.

- --env_reward_interval 5 and --reward_interval 10: environment and DDS reward cadences.
- --disable_auto_reset: prevents termination-triggered task resets; manual DDS resets still apply.

Invalid or unrelated combinations

- Do not use --no_render; it disables the camera updates required by Quest.
- Do not use --replay_data; Quest mode is a live DDS mode.
- Do not use Isaac AppLauncher's --xr; it starts Isaac Sim's native XR experience, not this integration.
- Do not add a conflicting --enable_dex1_dds, --enable_dex3_dds, or --enable_inspire_dds flag.
- Do not override the automatically selected action source, robot type, or required cameras.

xr_teleoperate run settings

Core choices

- --input-mode=hand|controller: default hand. Dex3 and Inspire use hand tracking; Dex1 can use either input mode.
- --display-mode=immersive|ego|pass-through: default immersive. Ego combines pass-through with a smaller first-person view; pass-through hides robot video.
- --arm=G1_29|H1_2: must match the simulator task.
- --ee=dex1|dex3|inspire_dfx: must match the simulator end effector.
- --img-server-ip=SIM_IP: simulator host address; upstream default 192.168.123.164 is only correct when it matches your network.
- --network-interface=IFACE: CycloneDDS interface. Specify it on multi-interface hosts.
- --sim: enables simulation behavior and is required here.
- --frequency=30: control and recording frequency; upstream default 30 Hz.

Optional modes

- --record: enables the s key recording workflow.
- --headless: runs xr_teleoperate without its local diagnostic display.
- --ipc: lets another program control xr_teleoperate state through IPC.
- --motion: upstream locomotion coexistence mode; not required for the fixed-base red-block task profiles in this guide.
- --task-dir, --task-name, --task-goal, --task-desc, and --task-steps: recording destination and metadata.

Display-mode guidance

- Use immersive for the lowest mental mapping cost during tabletop work.
- Use ego when the operator needs room awareness while retaining the simulated first-person view.
- Use pass-through only when camera video is deliberately unnecessary; it is not a camera-pipeline verification mode.

The simulator defaults to a 100 Hz control loop while xr_teleoperate defaults to 30 Hz command production. The simulator consumes the most recent shared-memory command each control step; this rate mismatch is intentional.

Network, ports, DDS topics, and shared memory

TCP/ZMQ and browser endpoints

- 60000 on simulator: teleimager configuration request/reply endpoint used by the image client.
- 55555 on simulator: head/front RGB stream from front_camera.
- 55556 on simulator: left wrist RGB stream from left_wrist_camera.
- 55557 on simulator: right wrist RGB stream from right_wrist_camera.
- 8012 on XR host: Vuer HTTPS/WSS endpoint reached by the Quest browser.
- 60001-60003: optional teleimager direct-WebRTC endpoints; intentionally disabled by --meta_quest.

Allow the XR host to reach ports 60000 and 55555-55557 on the simulator. Allow the Quest to reach port 8012 on the XR host. DDS also needs multicast/unicast traffic on the selected interface; it is not represented by one stable application port in this guide.

DDS domain and topics

All participants use Unitree channel/domain 1 in this integration.

- Humanoid arm/body command: rt/lowcmd; state: rt/lowstate.
- Dex1 commands: rt/dex1/left/cmd, rt/dex1/right/cmd; states use the same paths ending in /state.
- Dex3 commands: rt/dex3/left/cmd, rt/dex3/right/cmd; states use the same paths ending in /state.
- Inspire DFX command: rt/inspire/cmd; state: rt/inspire/state.
- Reset command: rt/reset_pose/cmd carrying a JSON String message.
- Simulator state: rt/sim_state; reward state: rt/rewards_state.

Simulator shared-memory boundary

DDS processes and the control loop exchange current values through named shared memory. Important buffers include dds_robot_cmd, isaac_robot_state, isaac_gripper_cmd/state, isaac_dex3_cmd/state, isaac_inspire_cmd/state, isaac_reset_pose_cmd, and isaac_sim_state. The camera writer uses a separate multi-image shared-memory buffer consumed by teleimager.

This split keeps network callbacks outside the real-time simulation loop: DDS workers handle serialization/networking, while the ActionProvider reads the latest local command without blocking on the network.

Reset behavior and randomized hospital semantics

The reset topic accepts categories through the String payload on rt/reset_pose/cmd:

- Category 1: object-only reset. The red block respawns collision-free on the current randomized table. The room, robot/table group, Ridgeback waiting transform, and tabletop layout remain the current layout.
- Category 2: full-scene reset. The shared full-reset path restores defaults, randomizes walls/furniture and the robot/table/Ridgeback group, places target/tabletop objects, resets Ridgeback joints/state machine, clears teleoperation timers, resets velocities, and preserves camera initialization behavior.

The randomized hospital task stores per-environment geometry in the room layout state. Robot pose/yaw, packing-table pose/yaw, target table-local pose, Ridgeback waiting/staging/delivery poses, selected wall layout, and tabletop placements share that source of truth. Teleoperation resets and Ridgeback assistance therefore follow the current layout instead of fixed world coordinates.

Manual reset diagnostics

The repository includes reset publishers for diagnostics. Run them only while the simulator is listening on the intended DDS domain and no physical robot shares that command environment.

```
# Inspect the script before use; it publishes to rt/reset_pose/cmd.
python reset_pose_test.py
```

For production Quest operation, prefer xr_teleoperate's normal simulated reset/recording flow or the task's native reset behavior so each reset is issued exactly once.

What to observe after a full hospital reset

- G1, the packing table, and Ridgeback move as one validated randomized group.
- Ridgeback returns to the randomized waiting pose behind G1 in robot-local coordinates.
- The red block and hospital props remain within the current table surface and out of reserved areas.
- Camera streams continue without a duplicate-reset startup stall.
- The assistant detects left/right hands in robot-local coordinates and delivers/returns relative to the current robot yaw.

Startup acceptance checklist

Before enabling control

- The simulator and XR host can ping one another by their intended LAN addresses.
- The Quest can browse to https://XR_HOST_IP:8012.
- No real robot is unintentionally present on DDS domain 1.
- The task and --arm/--ee profile match exactly.
- The simulator logs [Meta Quest] enabled cameras, ZMQ video with the expected robot and hand.
- front_camera, left_wrist_camera, and right_wrist_camera report ready at 480x640.
- Teleimager starts without a port-bind error and exposes ports 60000 and 55555-55557.
- The expected Dex1, Dex3, or Inspire DDS endpoint registers.
- The humanoid low-command/low-state endpoint and DDS ActionProvider start.
- The Quest shows a live first-person image, not a static first frame.
- Operator arms are aligned to the robot's initial pose before pressing r.

Five-minute functional check

1. Move the head slowly and confirm the Vuer presentation remains responsive.
2. Move one arm at a time and verify correct left/right mapping.
3. Open/close each Dex1, Dex3, or Inspire end effector.
4. Approach the red block without contact and verify camera latency remains controllable.
5. Perform one object-only reset; confirm the block returns to the current table.
6. Perform one full reset; for the hospital task confirm the complete group changes to a valid layout.
7. Execute one left-hand and one right-hand pick/place attempt.
8. For Ridgeback-enabled operation, verify staging, selected-hand delivery, basket success, and return.
9. If recording, save a short episode and verify files and disk growth.

Stop immediately if the simulated robot jumps at enable time, left/right controls are crossed, commands persist after xr_teleoperate exits, or DDS traffic reaches an unintended device.

Troubleshooting

Quest cannot open Vuer or WebSocket will not connect

- Confirm xr_teleoperate is running and listening on XR-host port 8012.
- Use the XR host's LAN IP in both URL positions, not the simulator IP and not localhost.
- Allow port 8012 through the XR-host firewall.
- Accept the self-signed certificate warning. Regenerate/copy cert.pem and key.pem if expired or misplaced.
- Avoid guest Wi-Fi/client isolation, which prevents Quest-to-host traffic.

xr_teleoperate cannot connect to image server

- Pass the simulator's LAN IP to --img-server-ip.
- Confirm ports 60000 and 55555-55557 are not blocked or already bound.
- Start the simulator before xr_teleoperate so teleimager is ready.
- Check that the selected task really uses --meta_quest and reports all three cameras.

Black, frozen, or stale camera frames

- Remove --no_render; use --headless for offscreen rendering.
- Keep camera_write_interval=1 while diagnosing.
- Verify the RTX renderer/GPU initializes and all cameras report ready.
- Reduce competing GPU workload. If tuning render cadence or JPEG quality, change one setting at a time and retest latency.
- pass-through display mode intentionally does not show the immersive robot view.

Robot moves but hand does not

- Confirm task and --ee match: Dex1=dex1, Dex3=dex3, Inspire=inspire_dfx.
- Use hand tracking for Dex3 and Inspire. Dex1 accepts controller or hand input.
- Check the corresponding DDS command/state topic appears in both simulator and xr_teleoperate logs.
- For Inspire, do not choose inspire_ftp; it uses an incompatible topic family.

No arm commands or intermittent DDS

- Pass the correct --network-interface to xr_teleoperate.
- Remove VPN/bridge ambiguity or explicitly configure CycloneDDS to the shared LAN interface.

- Ensure processes share DDS domain/channel 1 and multicast is permitted.
- Verify --sim is present on xr_teleoperate and action_source=dds is reported by the simulator.

Asset or scene startup errors

- Run . fetch_assets.sh and verify robot, room, object, and material USD files under assets/.
- Both drawer tasks currently emit a non-blocking MDL compiler error for the small-warehouse traffic-cone material OmniUe4Base_1. The material falls back; the scene, cameras, DDS, and control loop still operate.
- Headless GLFW/display warnings may be harmless if the RTX renderer, cameras, and loop start successfully. Treat camera initialization failure as blocking.

Validation and test commands

Static Quest profile tests

```
python -m unittest tests.test_meta_quest_redblocks
```

These tests cover all seven registered red-block task mappings, required camera names, ZMQ-only transport, and rejection of --no_render, replay mode, unsupported tasks, and conflicting hand choices.

Bounded simulator startup test

```
python sim_main.py --device cuda:0 --headless --meta_quest --max_steps 25 \
  --task Isaac-PickPlace-RedBlock-G129-Dex1-Joint
```

A successful smoke test constructs the stage and robot, renders all three cameras, starts teleimager and DDS, creates the action provider, completes 25 steps, prints the bounded-stop message, and exits cleanly.

Geometry regression test

```
python isaac-projects/room_randomizer_lab/test_placement.py
```

The randomized hospital validation covers 1,000 deterministic layouts, room/furniture collision constraints, Ridgeback waiting/staging/delivery bounds and corridors, tabletop bounds and overlap rules, reserved regions, target respawn on the current table, full-reset layout changes, and seed reproducibility.

Simulator-side runtime result, 2026-08-21

- GUI-capable CUDA/Vulkan environment and headless offscreen mode were available.
- Every task in the profile matrix started and exited successfully under --meta_quest --max_steps.
- Cameras were ready at 480x640 and ZMQ publishers were active on 55555-55557.
- Matching Dex1, Dex3, or Inspire endpoints plus humanoid low-command/low-state and DDS ActionProvider were created.
- A physical Meta Quest/xr_teleoperate peer was unavailable. Complete hardware sign-off still requires the five-minute checklist plus sustained operation, reset, recording, left/right hand, and Ridgeback scenarios on the deployment network.

Quick-reference run card

Terminal 1 — simulator host

```
cd /path/to/core_unitree_sim_isaaclab
python sim_main.py --device cuda:0 --headless --meta_quest \
  --task Isaac-PickPlace-RedBlock-Hospital-G129-Dex1-Joint
```

Terminal 2 — XR host

```
conda activate tv
cd ~/xr_teleoperate/teleop
python teleop_hand_and_arm.py --input-mode=controller \
  --arm=G1_29 --ee=dex1 --sim --record \
  --img-server-ip=SIM_IP --network-interface=IFACE
```

Meta Quest browser

```
https://XR_HOST_IP:8012/?ws=wss://XR_HOST_IP:8012
```

Accept the certificate warning, select Virtual Reality, allow permissions, align arms, then press r in Terminal 2. Press s to start/stop recording and q to quit.

Correct profile in one line

- G1 Dex1: G1_29 + dex1; controller or hand.
- G1 Dex3: G1_29 + dex3; hand only.
- G1 Inspire: G1_29 + inspire_dfx; hand only.
- H1-2 Inspire: H1_2 + inspire_dfx; hand only.

Source references

- Repository implementation: sim_main.py, tools/meta_quest.py, teleimager/src/teleimager/image_server.py, dds/, and tests/test_meta_quest_redblocks.py.
- Unitree xr_teleoperate: https://github.com/unitreerobotics/xr_teleoperate
- Unitree SDK2 Python: https://github.com/unitreerobotics/unitree_sdk2_python

- Upstream information was checked on 2026-08-21; re-check upstream release notes when upgrading xr_teleoperate.

Rebuild this PDF

```
python tools/build_meta_quest_pdf.py
```

The reviewed source is infodocs/meta_quest_redblock_operator_guide.md; the generated artifact is infodocs/Meta_Quest_RedBlock_Operator_Guide.pdf.